AdaptiveFlow Software Engineering Lab

Release 2 — Framework, Knowledgebase & Prototypes

CS212P – Software Systems and Tools Laboratory

Project Overview

AdaptiveFlow is a developer-driven, architecture-first framework for building self-optimizing desktop and web applications. The system observes user behavior using a lightweight, append-only tracker, persists usage statistics in a local database, and periodically evaluates feature utility through a background cycle to enable or disable features via a flat JSON configuration file. No runtime analysis, no code deletion, no parallel heavyweight monitors.

Member	Roll No.
Pranav Tej	CS24B057
Ankit Sinha	CS24B056
Karthik	CS24B046
Bhargav	CS24B028
Nilanjan Singh	CS24B029

Indian Institute of Technology Madras — April 2026

Contents

1 Problem Statement **3**
- 1.1 The Static UI Architecture Problem . . . 3
- 1.2 Why Existing Solutions Are Insufficient . . . 3

2 Proposed Solution **4**
- 2.1 Core Philosophy: Starve, Don't Delete . . . 4
- 2.2 The Four Core Modules . . . 4
  - 2.2.1 Development Guidelines (The Foundation) . . . 4
  - 2.2.2 Statistics and Tracking Module (The Observer) . . . 4
  - 2.2.3 The Flipper Module (The Gatekeeper) . . . 5
  - 2.2.4 The Weekly Adaptation Cycle (The Evaluator) . . . 5
- 2.3 The Feature ID Contract . . . 5
- 2.4 Deliberate Trade-offs . . . 5

3 The Knowledgebase: An Agent-Optimized Architecture **6**
- 3.1 Directory Structure . . . 6
- 3.2 Folder-Level Purposes . . . 7
- 3.3 Agent Traversal Protocol: O(1) Context Scaling . . . 7
- 3.4 The Router as Orchestrator . . . 8
- 3.5 The Memory and Context System . . . 8
- 3.6 Technologies and Stack Coverage . . . 8

4 Prototype Case Studies **9**
- 4.1 Prototype A: NEXUS Baseline (Without Knowledgebase) . . . 9
  - 4.1.1 Application Overview . . . 9
  - 4.1.2 System Architecture . . . 10
  - 4.1.3 Core Feature Modules . . . 10
  - 4.1.4 Heavy Background Work . . . 11
  - 4.1.5 Why This Is the Baseline, Not the Goal . . . 12
- 4.2 Prototype B: NEXUS Adaptive (With Knowledgebase) . . . 12
  - 4.2.1 Why Adaptability Matters for NEXUS . . . 12
  - 4.2.2 Feature Registry . . . 12
  - 4.2.3 Adaptive Metadata Schema . . . 13
  - 4.2.4 Adaptation Layers . . . 14
  - 4.2.5 Adaptive Workflow . . . 15
  - 4.2.6 Flipper Implementation . . . 16
  - 4.2.7 Weekly Evaluator Pseudocode . . . 16
- 4.3 Prototype C: Conduit RealWorld App (Retrofit with Knowledgebase) . . . 17
  - 4.3.1 Application Overview . . . 17
  - 4.3.2 Retrofitting Strategy . . . 17
  - 4.3.3 Feature Registry for Conduit . . . 17
  - 4.3.4 Per-User Schema Model . . . 18
  - 4.3.5 Usage Tracking Pipeline . . . 18
  - 4.3.6 Database Schema for Usage Statistics . . . 19
  - 4.3.7 Adaptation Cycle Trigger . . . 19
  - 4.3.8 Dual-Layer Runtime Enforcement . . . 19

4.3.9 Recovery Interface 20

5 System Architecture Comparison **21**

6 Tech Stack **21**
6.1 Knowledgebase (Stack-Agnostic Documentation) 21
6.2 Prototype B: NEXUS Adaptive (Primary Desktop App) 22
6.3 Prototype C: Conduit Retrofit (Full-Stack Web App) 23

7 Team Roles **23**

8 AI Usage Declaration **25**
8.1 Ankit 25
8.2 Pranav 25
8.3 Karthik 26
8.4 Bhargav 26
8.5 Nilanjan 27

9 Prompts **27**
9.1 Ankit 27
9.2 Pranav 30
9.3 Karthik 30
9.4 Bhargav 31
9.5 Nilanjan 32

10 Milestones & Timeline **34**

11 Goals for Final Release **34**
11.1 Framework Goals 34
11.2 Prototype Goals 35
11.3 Research Goals 35

I. Problem Statement

Desktop productivity applications accumulate features aggressively over their lifetimes. A modern spreadsheet editor or code-integrated tool may ship with upward of 500 individual commands; telemetry consistently shows that a single user interacts with fewer than 10% of these daily. The remaining 90%—the *bloat*—does not simply sit inert. Its effects are concrete and measurable.

A. The Static UI Architecture Problem

The "one-size-fits-all" static user interface creates distinct, overlapping pain-points for three classes of stakeholders.

Novice and junior users face disproportionate cognitive load. When every feature is always visible, beginners spend 10–15 minutes per day scanning cluttered toolbars simply to locate basic operations. The resulting feature blindness induces a steep and demoralising learning curve that has no functional justification.

Power users suffer from increased task completion time. Specialists with tightly scoped workflows must navigate across multiple tabs dozens of times each session to access the small cluster of tools that comprise their entire working repertoire. HCI research cited in our earlier proposal validates that forcing users to adapt to static software reduces engagement by up to 31% and decreases task efficiency by 22% compared to systems that dynamically match the user's behavior.

System hardware, particularly resource-constrained machines (4 GB RAM laptops, entry-level desktops), bears the most technically severe consequence. Modern UI frameworks load the entire DOM and initialize every backend module at startup regardless of whether the user will ever invoke them. Background sync workers, analytics daemons, and heavy computation threads run indefinitely once the application boots, consuming RAM and CPU for features the user may visit once a month or never.

B. Why Existing Solutions Are Insufficient

Standard workarounds fail in characteristic ways.

- **Manual customization tools** (e.g., Microsoft's Quick Access Toolbar) require explicit user configuration. Industry data shows fewer than 5% of users actively customize their interface; the time and cognitive overhead of customization defeats its own purpose.
- **CSS-based visibility hiding** (`display:none`) removes elements from the rendered view but does not prevent them from being loaded into memory, initialized, or executing background work.
- **Automated runtime debloaters** and static code-deletion tools introduce what we term the *Observer Paradox*.

The Observer Paradox. A background monitoring process that watches application behavior and infers unused code paths—whether a Python AST parser, a real-time WebSocket telemetry pipeline, or a 500 ms polling profiler—can itself consume 150–400 MB of RAM and significant CPU cycles. The cost of observation exceeds the cost of the bloat it is

attempting to eliminate. Furthermore, automated code-deletion tools permanently remove code paths, silently severing dependencies that are not immediately visible to the static analysis tool and causing hard-to-diagnose regressions.

The fundamental problem is not that adaptation is impossible; it is that every prior approach attempts adaptation through a mechanism that contradicts the goal.

II. Proposed Solution

A. Core Philosophy: Starve, Don't Delete

AdaptiveFlow rejects the parallel observer in favor of a two-part design contract:

1. **Developer-enforced architectural discipline** applied at build time, codified as a portable knowledgebase of guidelines.
2. **A flat JSON boolean configuration file** read exactly once at startup, evaluated periodically in the background, and consulted by an in-memory lookup engine during runtime.

The result is zero parallel processing overhead and 100% application stability. No code is deleted from disk. Instead, unused features are *starved* of active compute resources—they are never loaded into RAM, never parsed by the JavaScript engine, never initialized by the class loader. To the operating system and to the user, those features do not exist at runtime, but a developer can restore any of them by flipping a boolean.

B. The Four Core Modules

Development Guidelines (The Foundation)

The framework depends entirely on the application being built to be *intrinsically prunable*. This requires three architectural disciplines.

Modular decoupling: Following the Command Pattern strictly. UI elements contain no business logic; they dispatch commands. Business logic is never entangled with view rendering or database access in the same function.

Lazy loading: Heavy libraries, background threads, and complex engines are never eagerly initialized at startup. They are loaded conditionally, only after the Flipper module confirms the feature is active.

Feature ID tagging: Every independently disableable capability is assigned a globally unique `ID_[DOMAIN]_[CAPABILITY]` string identifier. This identifier is the contract between the UI layer, the tracking system, the usage database, and the configuration schema. A mismatch at any point silently breaks the adaptation cycle for that feature.

Statistics and Tracking Module (The Observer)

A hyper-lightweight tracking mechanism built directly into the application's event loop. It uses passive event delegation on tagged UI elements, creates a minimal two-field payload (`f_id`, `ts`), pushes it to an in-memory queue in O(1), and flushes the queue asynchronously

on a timer or lifecycle event. It adds sub-millisecond overhead per interaction and never performs synchronous I/O on the main thread. No AST parsers, no runtime profilers, no real-time sockets.

The Flipper Module (The Gatekeeper)

The Flipper reads the flat feature-flag JSON schema once at application startup and caches all flags in an in-memory dictionary. At runtime, every feature gate in the codebase queries `isActive(featureId)`, which performs a single dictionary lookup—O(1), synchronous, with no disk or network access. Unknown feature IDs default to `true` to prevent accidental lockout of unregistered features. The Flipper state is immutable during a user's active session, preventing the UI from shifting beneath a user's hands while they are working.

The Weekly Adaptation Cycle (The Evaluator)

A background function—triggered on a cron schedule, on login, or after a fixed number of boots—queries the usage statistics store for the rolling 14-day usage window for each feature. Features that fall below a predefined threshold are flipped to `false` in the JSON schema. Locked safety features (navigation, authentication, settings, the restore UI) are never evaluated and can never be set to `false`. Users may manually restore any hidden feature from the Settings page at any time.

C. The Feature ID Contract

The Feature ID is the single connective tissue linking all four modules. Its format is `ID_SCREAMING_SNAKE_CASE`. Because the four modules are deliberately decoupled and never communicate directly at runtime, there is no mechanism to detect a mismatch. A typo in a Feature ID does not throw a runtime exception—it silently creates a ghost entry in the statistics database that accumulates no usage, while the Flipper treats the mistyped key as `undefined` and defaults to `true`. The feature is never adapted. This is why the Feature Tagging audit is treated as a strict prerequisite: it must be completed and committed to version control before any implementation begins.

D. Deliberate Trade-offs

- **No disk footprint reduction.** All feature code remains on the hard drive. The framework targets active compute (RAM and CPU), not static storage.
- **High engineering discipline required.** This is not a "plug-and-play" script. It requires thoughtful feature tagging, architectural decoupling, and lazy loading at every relevant entry point.
- **No real-time adaptation.** The Flipper state is fixed for the duration of a session. The adaptation cycle is intentionally periodic.
- **Not suitable for legacy monoliths without partial refactoring.** Tightly coupled code that interweaves UI, business logic, and database calls cannot be meaningfully gated without first separating those concerns.

III. The Knowledgebase: An Agent-Optimized Architecture

The primary deliverable of Release 2 is not just working code—it is a *reusable, agent-traversable knowledgebase* that enables any AI coding agent (or human developer) to implement the adaptive framework correctly and efficiently on any target codebase. The knowledgebase is structured to prevent context-window bloat by enforcing a deterministic, sequential file traversal protocol.

A. Directory Structure

```
adaptive_software_kb/
|
|-- 00_Context_and_Goals/
|   |-- 00_goal.md            # Core philosophy & entry point
|   `-- 01_developer_context.md # Context intake & project profiling
|
|-- 01_Architecture/
|   |-- 01_router.md          # Routing orchestrator: 5-category matrix
|   |-- 02_flipper_module.md  # Flipper runtime + tracking + weekly adaptation
|   `-- 03_metafile_schema.md # Feature flag config contract & persistence rules
|
|-- 02_Assessment_and_Tagging/
|   `-- 01_feature_tagging.md # Feature audit & ID assignment
|
|-- 03_Code_Patterns/
|   |-- 01_frontend_dom.md    # UI component gating (React/Vue/DOM)
|   |-- 02_backend_api.md     # API route gating (Express/FastAPI)
|   |-- 03_background_jobs.md # Worker/cron gating
|   |-- 04_middleware.md      # Request-level gate middleware
|   |-- 05_asset_manager.md   # Heavy asset preload control
|   |-- 06_db_query_logic.md  # Query fragmentation (SQL/NoSQL)
|   |-- 07_package_imports.md # Heavy package lazy-import
|   `-- 08_memory_cleanup.md  # Post-removal leak prevention
|
`-- 04_Memory_and_Context/
    |-- 00_memory_protocol.md
    |-- 01_Working_Memory/
    |   |-- 01_active_task_state.md
    |   |-- 02_inter_agent_scratchpad.md
    |   `-- 03_action_log.md
    |-- 02_Orchestrator_Learnings/
    |   `-- 01_routing_heuristics.md
    `-- 03_Subagent_Learnings/
        |-- 01_codebase_quirks.md
        |-- 02_error_ledger.md
        `-- 03_codebase_map.md
```

Listing 1: Knowledgebase directory layout

B. Folder-Level Purposes

Folder	Purpose
`00_Context_and_Goals`	Entry point. Agent reads the core philosophy, then runs the developer context intake to build a Project Context Summary that every downstream agent consumes.
`01_Architecture`	Core framework documentation: the Flipper engine (runtime gating and weekly adaptation cycle), the metadata schema contract, and the routing orchestrator that directs agents to the correct implementation file. Three files cover all architectural concerns: `02_flipper_module.md` consolidates the former monitor, statistics, and flipper-logic files into a single authoritative module specification.
`02_Assessment_and_Tagging`	Feature audit instructions. Produces the Feature Registry, the initial Metadata Schema, and the implementation Tagging Checklist. Must be completed before any implementation work begins.
`03_Code_Patterns`	Eight implementation templates, one per architectural layer. Each file contains stack-specific code examples, anti-patterns, and an "Observed In" section with real-world application examples drawn from the three prototypes.
`04_Memory_and_Context`	Agent state system following ICM (In-Context Memory) principles. Contains working memory (active task state, inter-agent scratchpad, action log) and long-term learnings (routing heuristics, codebase quirks, error ledger).

Table 1: Knowledgebase folder purposes

C. Agent Traversal Protocol: O(1) Context Scaling

The knowledgebase is explicitly designed to prevent agents from speculatively loading irrelevant files. The enforced traversal protocol is:

1. **Entry through `00_goal.md`.** This file explains the Observer Paradox, the four modules, and the Feature ID contract. It ends with a hard directive: do not read any other file yet. Proceed to the Router.

2. **Context intake via `01_developer_context.md`.** The context agent resolves seven project dimensions (application type, frontend stack, backend stack, database, identity model, performance constraints, and codebase state) and produces a structured Project Context Summary JSON that is attached to all downstream agent calls.

3. **Routing via `01_router.md`.** The Router classifies the developer's request into one of five categories (core infrastructure, assessment, frontend, backend/data, memory/orchestration) and outputs the minimal set of files the agent must read—nothing more.

4. **Targeted implementation.** The agent reads only the routed files, then generates code. It appends outcomes to the action log before handoff.

This architecture guarantees that an agent implementing a frontend UI gate never reads the backend API gating file. An agent auditing a new codebase never reads the memory cleanup patterns. Token consumption scales with task complexity, not with knowledgebase size.

D. The Router as Orchestrator

`01_router.md` functions as the central nervous system of the knowledgebase. Its routing matrix maps trigger keywords to precise file targets:

Category	Routed To
Core infrastructure (setup, flipper, schema, weekly cycle)	`02_flipper_module.md`, `03_metafile_schema.md`
Assessment and tagging (audit, feature IDs, codebase map)	`02_Assessment_and_Tagging/` `01_feature_tagging.md`
Frontend (UI, DOM, component, assets, imports, leaks)	`01_frontend_dom.md`, `05_asset_manager.md`, `07_package_imports.md`, `08_memory_cleanup.md`
Backend/data (API, SQL, cron, worker, middleware)	`02_backend_api.md`, `03_background_jobs.md`, `04_middleware.md`, `06_db_query_logic.md`
Orchestration state (handoff, resume, action log)	`04_Memory_and_Context/*`

Table 2: Router classification matrix

E. The Memory and Context System

The `04_Memory_and_Context` folder implements an agent memory layer that prevents repeated work and preserves state across multi-session builds.

Working Memory captures the current active task state, the inter-agent scratchpad for raw handoff payloads, and the action log that records every completed step with its outcome. Before routing any new task, the orchestrator reads the action log to avoid re-running completed steps.

Long-Term Learnings are maintained as two categories of accumulated knowledge: *Orchestrator Learnings* (routing heuristics derived from past sessions) and *Subagent Learnings* (codebase-specific quirks, error ledger entries, and the codebase map). These accumulate over the project lifetime and reduce per-session setup overhead.

F. Technologies and Stack Coverage

The knowledgebase is intentionally stack-agnostic. It explicitly supports the following environments through parameterised implementation templates.

- **Frontend:** React (TypeScript), Vue, Vanilla JS/DOM
- **Backend:** Express.js, FastAPI, Django, Spring Boot
- **Database:** PostgreSQL, MySQL, SQLite, MongoDB, DynamoDB

- **ORM:** Prisma, TypeORM, SQLAlchemy, Sequelize, Mongoose
- **Platform:** Web (SPA and SSR), Desktop (Electron), Mobile (React Native, iOS, Android)
- **Feature gate mechanisms:** JSON boolean config, in-memory singleton, optional Redis-backed distributed flipper, weekly adaptation cron

IV. Prototype Case Studies

Release 2 delivers three distinct prototype implementations, each serving a different validation purpose within the overall research framework.

Prototype	Codebase	Validation Purpose
A	NEXUS (scratch, no KB)	Baseline: a feature-rich application built without adaptive guidelines to establish the pre-adaptation state.
B	NEXUS (scratch, with KB)	Primary: the same application rebuilt with full adaptive integration using the knowledge-base. Demonstrates framework correctness and resource savings.
C	Conduit RealWorld (existing repo, with KB)	Retrofit: the framework applied to an existing, independently authored full-stack application. Demonstrates generalisability.

Table 3: Prototype matrix and validation purposes

A. Prototype A: NEXUS Baseline (Without Knowledgebase)

Application Overview

NEXUS is a premium, full-stack desktop productivity and financial intelligence application. It is intentionally feature-heavy and architecturally layered, with every major subsystem independently operable and clearly modular. It was constructed as the primary testbed for the adaptive framework: feature-rich enough that adaptation produces measurable savings, modular enough that gating can be applied cleanly.

NEXUS bridges the gap between a web application and a native desktop application through a split architecture running entirely locally.

System Architecture

```
+---------------------------------+
| Frontend: React + Vite          |
| Glassmorphic UI, CSS tokens     |
+------------------+--------------+
                   | IPC Bridge
+------------------+--------------+
| Backend: Express (port 4771)    |
| Validation, complex transactions|
| Worker thread orchestration     |
+------------------+--------------+
                   | Prisma ORM
+------------------+--------------+
| Database: SQLite (nexus.db)     |
| Local filesystem storage        |
| Zero cloud dependency           |
+---------------------------------+
```

Listing 2: NEXUS baseline architecture layers

Frontend (React + Vite): A highly responsive glassmorphic UI using custom CSS design tokens defined in `index.css`. The component architecture is modular, with each major feature area contained in a dedicated page component and lazy-loaded from the router.

Backend (Express): An internal API running on a dynamic port (default 4771), responsible for data validation, complex financial transactions, and orchestrating worker threads. The Express layer is the sole interface between the React frontend and the Prisma/SQLite layer.

Database (Prisma + SQLite): A local `nexus.db` file stored in the user's `userData` directory in production. Prisma provides type-safe query access and `$transaction` guarantees for double-entry ledger operations that atomically update account balances when transactions are created.

Background Workers (worker_threads): Heavy computational tasks are completely offloaded from the main Node.js process and the UI render thread, preventing UI freezing during large-scale data operations.

Core Feature Modules

NEXUS is designed as a unified command center with ten interwoven modules.

Module	**Description**
Dashboard	Central hub. Lazy-loads a Recharts financial graph. Displays recent transactions, upcoming tasks, and quick-summary widgets.
Accounts & Transactions	Double-entry style ledger. Creating a transaction atomically adjusts the associated account balance using `$transaction` to prevent race conditions.
Tasks & Time Tracking	Priority-grouped task manager (TODO, IN_PROGRESS, DONE) with active duration logging.
Habits	Consistency tracking grid for building recurring behavioral habits.
Notes & Journal	Sidebar notes with tagging and a daily journal with mood metadata.
Budgets & Goals	Milestone tracking tied to categories or specific financial targets.
Investments	Portfolio tracker that dynamically updates overall Net Worth.
Reports (Worker)	Offloaded to `report.worker.ts`. Aggregates transactions, accounts, and investments across a user-specified date range. Computes cash flow, categorises income/expenses, and calculates real-time Net Worth.
Analytics (Worker & Cron)	Triggered weekly (Sunday midnight) or on demand. Computes three cross-domain performance scores (Financial, Productivity, Habit Consistency) by querying the Transaction, Task, and HabitLog tables. Persists a new AnalyticsData record. The UI fetches the latest computed record.
Feature Usage Tracking	Aggregates FeatureUsageLog telemetry asynchronously through the general worker to maintain a clean database without blocking standard API endpoints.

Table 4: NEXUS module inventory

Heavy Background Work

The Reports Worker (`report.worker.ts`) performs extensive multi-table data aggregation across `Transactions`, `Accounts`, and `Investments`. It fetches thousands of transaction records within a date range, categorises them into INCOME and EXPENSE, and calculates cash flow. It simultaneously aggregates all account balances and computes real-time Net Worth by multiplying investment quantities by their unit prices. By computing this off the main thread, the Electron UI and the Express router remain instantly responsive even when the user has years of financial data.

The Analytics Worker (`analytics.worker.ts`) computes three primary scores:

$$\text{Financial Score} = \frac{\text{Total Income} - \text{Total Expense}}{\text{Total Income}} \times 100$$

$$\text{Productivity Score} = \frac{\text{Completed Tasks}}{\text{Total Tasks}} \times 100$$

$$\text{Habit Consistency} = \frac{\text{Completed Habits}}{\text{Total Habits}} \times 100$$

The general CRON (`cron.ts` and `worker.ts`) handles daily budget threshold evaluations and habit log resets, and aggregates FeatureUsageLog telemetry for the adaptation evaluation pipeline.

Why This Is the Baseline, Not the Goal

Prototype A establishes the pre-adaptation state: all features are always loaded, all workers always start, the analytics cron always runs regardless of whether the user has ever opened the analytics module. On constrained hardware this represents peak resource consumption. It serves as the control against which the adaptive version (Prototype B) is measured.

B. Prototype B: NEXUS Adaptive (With Knowledgebase)

Why Adaptability Matters for NEXUS

No two NEXUS users have the same workflow. A student uses tasks, notes, habits, and time tracking. A freelancer uses accounts, transactions, reports, and time tracking. A household user uses budgets, goals, and journals. A feature-rich application should not force every user profile to carry the same interface and background processing workload. Prototype B applies the full adaptive framework to NEXUS, observing feature usage and adjusting the active runtime feature set at every layer of the architecture.

Feature Registry

The Feature Tagging audit produced the following registry for NEXUS. Feature IDs are divided into locked (always-on safety features) and adaptive (usage-evaluated) categories.

Locked features are always enabled because they are necessary for navigation, safety, and the recovery path:

```
ID_NAV_SIDEBAR
ID_APP_SETTINGS
ID_RESTORE_FEATURES_TAB
ID_ERROR_BOUNDARY
ID_DASHBOARD_VIEW
ID_USAGE_TRACKING
ID_ADAPTATION_WEEKLY_EVALUATOR
```

Listing 3: NEXUS locked feature IDs

Adaptive feature modules are grouped by domain:

Domain	Feature IDs	Threshold
Finance	ID_FINANCE_ACCOUNTS, ID_FINANCE_TRANSACTIONS, ID_FINANCE_BUDGETS, ID_FINANCE_INVESTMENTS	2–3
Productivity	ID_PRODUCTIVITY_TASKS, ID_PRODUCTIVITY_HABITS, ID_PRODUCTIVITY_NOTES, ID_PRODUCTIVITY_GOALS, ID_PRODUCTIVITY_TIME_TRACKING, ID_PRODUCTIVITY_JOURNAL	2–3
Dashboard Widgets	ID_DASHBOARD_FINANCIAL_SUMMARY, ID_DASHBOARD_RECENT_TRANSACTIONS, ID_DASHBOARD_UPCOMING_TASKS, ID_DASHBOARD_GOAL_OVERVIEW, ID_DASHBOARD_JOURNAL_OVERVIEW	1–2
Heavy Frontend	ID_DASHBOARD_ANALYTICS_CHART	1
Reports	ID_REPORTS_GENERATE, ID_REPORTS_EXPORT_JSON	1
Analytics	ID_ANALYTICS_SCORE_VIEW, ID_ANALYTICS_WEEKLY_COMPUTE	1
Background Jobs	ID_SYNC_PERIODIC, ID_BUDGET_DAILY_CHECKS, ID_USAGE_WEEKLY_SUMMARY	0 (manual)

Table 5: NEXUS adaptive feature registry summary

Features with a threshold of 0 are manually controlled only and are not automatically disabled by the weekly evaluator.

Adaptive Metadata Schema

The feature state is stored as a flat JSON schema file (`adaptive_config/feature-flags.local.json`):

```
{
  "schemaVersion": "1.0",
  "userId": "local-profile",
  "lastEvaluated": "1970-01-01T00:00:00.000Z",
  "lockedFeatures": [
    "ID_NAV_SIDEBAR",
    "ID_APP_SETTINGS",
    "ID_RESTORE_FEATURES_TAB",
    "ID_ERROR_BOUNDARY",
    "ID_DASHBOARD_VIEW",
    "ID_USAGE_TRACKING",
    "ID_ADAPTATION_WEEKLY_EVALUATOR"
  ],
  "features": {
    "ID_PRODUCTIVITY_TASKS":              true,
    "ID_PRODUCTIVITY_HABITS":             true,
    "ID_PRODUCTIVITY_NOTES":              true,
    "ID_PRODUCTIVITY_GOALS":              true,
    "ID_PRODUCTIVITY_TIME_TRACKING":      true,
    "ID_PRODUCTIVITY_JOURNAL":            true,
```

```
    "ID_FINANCE_ACCOUNTS":                true,
    "ID_FINANCE_TRANSACTIONS":            true,
    "ID_FINANCE_BUDGETS":                 true,
    "ID_FINANCE_INVESTMENTS":             true,
    "ID_DASHBOARD_FINANCIAL_SUMMARY":     true,
    "ID_DASHBOARD_RECENT_TRANSACTIONS":   true,
    "ID_DASHBOARD_UPCOMING_TASKS":        true,
    "ID_DASHBOARD_GOAL_OVERVIEW":         true,
    "ID_DASHBOARD_JOURNAL_OVERVIEW":      true,
    "ID_DASHBOARD_ANALYTICS_CHART":       true,
    "ID_REPORTS_GENERATE":                true,
    "ID_REPORTS_EXPORT_JSON":             true,
    "ID_ANALYTICS_SCORE_VIEW":            true,
    "ID_ANALYTICS_WEEKLY_COMPUTE":        true,
    "ID_SYNC_PERIODIC":                   true,
    "ID_BUDGET_DAILY_CHECKS":             true,
    "ID_USAGE_WEEKLY_SUMMARY":            true
  }
}
```

Listing 4: NEXUS adaptive metadata schema

Schema invariants: the `features` object is one-level flat for O(1) lookup. `lockedFeatures` entries are always forced `true` by the schema sanitization routine. Missing or unknown IDs default to `true` at runtime (fail-open). Writes are performed via a temporary-file-and-rename strategy to minimize the risk of corruption during power loss.

Adaptation Layers

The adaptive version gates features at every architectural layer.

Layer	What Adapts	Key Files
Feature registry	Feature IDs, categories, thresholds, locked status	`src/adaptive/registry.ts`
Startup bootstrap	Loads schema before app renders	`src/adaptive/bootstrap.ts`
Frontend flipper	O(1) client-side checks, React context	`src/adaptive/flipper.ts`, `AdaptiveProvider.tsx`
Route gating	Full page modules (Tasks, Reports, Analytics)	`src/App.tsx`, `FeatureRoute.tsx`
Sidebar gating	Navigation item visibility	`src/components/Layout.tsx`
Widget gating	Dashboard data fetches and UI sections	`src/pages/Dashboard.tsx`
Heavy import gating	Recharts lazy-load	`src/components/HeavyChart.tsx`
Usage tracking	Batched invocation events	`src/adaptive/featureTracker.ts`
API gating	Backend route suites	`server/app.ts`, `server/adaptive/middleware.ts`
Reports & workers	Worker thread spawning for reports and analytics	`server/routes/reports.ts`, `electron/workers/runWorker.ts`
Cron & background	Budget checks, analytics, adaptive evaluation	`server/cron.ts`, `server/worker.ts`
Restore controls	Manual feature recovery from Settings UI	`src/pages/Settings.tsx`, `HiddenFeatureNotice.tsx`

Table 6: NEXUS adaptive layer mapping

Adaptive Workflow

The lifecycle of adaptation in NEXUS follows a deterministic sequence:

1. Application starts. It reads `adaptive_config/feature-flags.local.json` from disk.
2. Both the client-side Flipper (`src/adaptive/flipper.ts`) and the server-side Flipper (`server/adaptive/flipper.ts`) are initialised with the loaded schema before any routes or UI components mount.
3. Only active routes, sidebar items, and dashboard widgets are rendered. Disabled page modules are never imported; their JavaScript bundles are never parsed by the engine.
4. As the user interacts with tagged features, the client-side tracker batches events (`f_id`, `ts`) into an in-memory queue.
5. The queue flushes to `POST /api/feature-usage/batch` on a timer or on tab-hidden/page-unload lifecycle events.
6. The backend upserts `FeatureUsageLog` rows in SQLite atomically, partitioned by day (time bucket).
7. The weekly adaptation worker queries the rolling 14-day usage window, compares each feature's total against its `adaptationThreshold`, and writes the updated schema back to disk.

8. The next application launch reflects the adapted feature set.
9. At any time, the user may open Settings, see a list of currently hidden features, and restore any of them with a single click.

Flipper Implementation

```typescript
class FlipperEngine {
  private flags: Record<string, boolean> = {};
  private initialized = false;

  init(schema: { features: Record<string, boolean> }): void {
    this.flags = schema.features ?? {};
    this.initialized = true;
  }

  isActive(featureId: string): boolean {
    if (!this.initialized) {
      console.warn("Flipper not initialized. Defaulting TRUE.");
      return true;
    }
    if (this.flags[featureId] === undefined) {
      console.warn(`Unknown FeatureID: ${featureId}. Defaulting TRUE.`);
      return true;
    }
    return this.flags[featureId];
  }

  getAllFlags(): Record<string, boolean> {
    return { ...this.flags };
  }

  overrideFlag(featureId: string, state: boolean): void {
    this.flags[featureId] = state;
  }
}

export const Flipper = new FlipperEngine();
```

Listing 5: Core Flipper engine (TypeScript)

Weekly Evaluator Pseudocode

```python
def run_adaptation_cycle(user_id):
    schema = read_metafile_schema(user_id)
    cutoff = datetime.now() - timedelta(days=14)

    recent_usage = db.query("""
        SELECT feature_id, SUM(usage_count) as total
        FROM feature_usage_stats
        WHERE user_id = ? AND time_bucket >= ?
        GROUP BY feature_id
    """, user_id, cutoff.date())

    for feature_id, current_state in schema["features"].items():
        if feature_id in schema["lockedFeatures"]:
```

```
            continue
        threshold = registry.get_threshold(feature_id)
        total = recent_usage.get(feature_id, 0)
        schema["features"][feature_id] = (total >= threshold)

    schema["lastEvaluated"] = datetime.now().isoformat()
    write_metafile_schema_atomic(user_id, schema)
```

Listing 6: Adaptation cycle evaluator

C. Prototype C: Conduit RealWorld App (Retrofit with Knowledgebase)

Application Overview

The Conduit RealWorld application is a publicly available, independently authored full-stack implementation of the Medium-style blogging platform specification. It provides:

- User authentication (register, login, logout, current user profile)
- Article publishing (create, read, update, delete, paginated listings)
- Social features: profiles, follow/unfollow, favorites, comments
- Discovery: tag-based filtering, global feed, personal follow-based feed

Tech stack: React + Vite frontend, Express.js backend, Sequelize ORM, PostgreSQL database. The codebase was sourced from the public GitHub repository `TonyMckes/conduit-realworld-`

This prototype is the critical validation case: the knowledgebase must be robust enough to guide correct adaptive integration into a codebase it was never designed around, written by a developer with no knowledge of the framework.

Retrofitting Strategy

The retrofit followed the knowledgebase traversal protocol strictly, beginning with the Feature Tagging audit before any implementation.

Repository structure after adaptation:

```
conduit-realworld-example-app/
|-- frontend/              # React + Vite UI
|-- backend/               # Express.js + Sequelize
|-- adaptive_config/       # New: governance artifacts
|   |-- feature_registry.json
|   |-- initial_metadata_schema.json
|   `-- tagging_checklist.md
```

Listing 7: Conduit adaptive repository layout

Feature Registry for Conduit

Locked features (always-on, never evaluated):

```
ID_NAV_TOPBAR
ID_AUTH_LOGIN
ID_AUTH_REGISTER
ID_AUTH_LOGOUT
```

```
ID_APP_SETTINGS
ID_RESTORE_HIDDEN_FEATURES
ID_ERROR_BOUNDARY
```

Listing 8: Conduit locked feature IDs

Adaptive features (usage-evaluated):

Feature ID	Display Name	Threshold	Category
`ID_DISCOVERY_POPULAR_TAGS`	Popular Tags Sidebar	3	Heavy UI component
`ID_FEED_PERSONAL`	Personal Feed Tab	3	Backend API + Frontend
`ID_PROFILE_FAVORITED_ARTICLES`	Favorited Articles Tab	2	Backend API + Frontend
`ID_ARTICLE_COMMENTS`	Article Comments Section	3	Backend API + Frontend

Table 7: Conduit adaptive feature registry

Per-User Schema Model

Because Conduit is a multi-user web application, the adaptive schema is stored *per user* rather than per machine. The `Users` Sequelize model was extended with an `adaptiveSchema` JSON column (Sequelize's `DataTypes.JSON`).

```
{
  "schemaVersion": "1.0",
  "userId": "<user-uuid>",
  "lastEvaluated": "1970-01-01T00:00:00.000Z",
  "lockedFeatures": [
    "ID_NAV_TOPBAR", "ID_AUTH_LOGIN",
    "ID_AUTH_REGISTER", "ID_AUTH_LOGOUT",
    "ID_APP_SETTINGS", "ID_RESTORE_HIDDEN_FEATURES",
    "ID_ERROR_BOUNDARY"
  ],
  "features": {
    "ID_DISCOVERY_POPULAR_TAGS":      true,
    "ID_FEED_PERSONAL":               true,
    "ID_PROFILE_FAVORITED_ARTICLES":  true,
    "ID_ARTICLE_COMMENTS":            true
  }
}
```

Listing 9: Per-user Conduit adaptive schema (initial state)

New users are initialised with all features `true`. The `lastEvaluated` timestamp is set to Unix epoch to force first-evaluation eligibility immediately. Schema sanitization (both frontend and backend) catches malformed or missing schemas and returns a safe default.

Usage Tracking Pipeline

The tracker (`frontend/src/adaptive/tracker.js`) listens for `document` click events on elements tagged with `data-feature-id`. Its control rules:

- **Throttle:** maximum one event per feature per 1000 ms (prevents rapid-fire click inflation)
- **Queue cap:** 500 events before forced flush
- **Flush interval:** every 30 seconds via `setInterval`
- **Lifecycle flush:** on tab hidden and before unload using `navigator.sendBeacon` with the `keepalive` flag to guarantee delivery during page close

Events are sent to `POST /api/adaptive/track`. If no authentication header exists, the queue is dropped—guest usage is not persisted, since the schema is per-user.

Database Schema for Usage Statistics

```sql
CREATE TABLE feature_usage_stats (
  user_id     VARCHAR(255)  NOT NULL,
  feature_id  VARCHAR(100)  NOT NULL,
  time_bucket DATE          NOT NULL,  -- e.g. '2026-04-14'
  usage_count INT           DEFAULT 1,
  PRIMARY KEY (user_id, feature_id, time_bucket)
);
```

Listing 10: Conduit feature usage statistics table

The composite primary key enables efficient upsert operations. The ingestion endpoint uses Sequelize's `upsert` (which maps to `INSERT ... ON CONFLICT DO UPDATE`) to atomically increment `usage_count` without read-modify-write race conditions.

Adaptation Cycle Trigger

Because Conduit has no always-running server-side scheduler, the adaptation cycle is triggered asynchronously (`setImmediate`) on two authenticated user flows: user login (`signIn`) and current-user fetch (`currentUser`). The evaluator checks whether the minimum 7-day re-evaluation gap has elapsed before running; if not, it returns immediately without touching the schema.

Dual-Layer Runtime Enforcement

Frontend gating uses the `FeatureGate` component and `useFlipper` hook:

- Popular tags sidebar is hidden when `ID_DISCOVERY_POPULAR_TAGS` is `false`
- Personal feed tab is removed and the feed defaults to global when `ID_FEED_PERSONAL` is `false`
- Favorited articles tab and its requests are disabled when `ID_PROFILE_FAVORITED_ARTICLES` is `false`
- Comments section is hidden when `ID_ARTICLE_COMMENTS` is `false`

Backend gating adds a middleware layer for all gated routes:

```go
// Personal feed requires ID_FEED_PERSONAL
router.get('/api/articles/feed',
  authenticate,
```

```
  flipperGuard('ID_FEED_PERSONAL'),
  ArticlesController.listFeedArticles
);

// Comments require ID_ARTICLE_COMMENTS
router.get('/api/articles/:slug/comments',
  optionalAuth,
  flipperGuard('ID_ARTICLE_COMMENTS'),
  CommentsController.getComments
);
```

Listing 11: Conduit backend route gating

When a disabled feature is accessed directly (bypassing the UI), the backend returns HTTP 403 with a structured error: { `"error": "Feature disabled", "featureId": "ID_FEED_PERSONAL"` }. The dual-layer design ensures that even a technically savvy user who manually crafts an API request cannot bypass a feature gate.

Recovery Interface

The `HiddenFeaturesManager` component lists all adaptive features currently set to `false`. Clicking "Restore" calls `overrideFlag` on the frontend Flipper and persists the change via `PUT /api/user` with the updated `adaptiveSchema` embedded in the request body. Locked features are excluded from the restore list—they cannot be disabled by the evaluator and therefore never appear as hidden.

V. System Architecture Comparison

Concern	**Prototype A** (NEXUS Baseline)	**Prototype B** (NEXUS Adaptive)	**Prototype C** (Conduit Retrofit)
Feature flags	None	Flat JSON, local file	JSON column in Users table
Schema scope	N/A	Per machine (local profile)	Per authenticated user
Flipper lookup	N/A	O(1) in-memory singleton	O(1) in-memory per request
Tracking	FeatureUsageLog via cron	Batched tracker, 30s flush	Batched tracker, 30s flush + sendBeacon
Evaluation trigger	N/A	Weekly cron (Sunday 00:00)	Login / currentUser (async, 7-day gap)
Evaluation window	N/A	14 days	14 days
Frontend gate	Always renders	FeatureRoute, FeatureGate, useFlipper	FeatureGate, useFlipper
Backend gate	Always executes	Middleware (flipperGuard)	Middleware (flipperGuard)
Worker control	Always spawns	Gated by Flipper prespawn	N/A (no workers)
Recovery UI	None	Settings page	HiddenFeaturesManager
Locked features	N/A	7 locked IDs	7 locked IDs

Table 8: Side-by-side architecture comparison of all three prototypes

VI. Tech Stack

A. Knowledgebase (Stack-Agnostic Documentation)

The knowledgebase itself is a collection of Markdown files. No runtime dependencies. It is designed to be checked into any project repository and read by AI coding agents or human developers.

B. Prototype B: NEXUS Adaptive (Primary Desktop App)

Layer	Technology	Justification
Desktop container	Electron.js	Wraps the web app as a native offline executable. Provides the IPC bridge for React-to-Node filesystem communication.
Frontend framework	React 18 + TypeScript + Vite	Component model perfectly suited for conditional rendering. Context API used for the global Flipper provider. Vite ensures fast development builds.
Backend runtime	Node.js + Express	Runs the MAPE-K adaptation logic, complex transactions, and worker orchestration outside the main UI render thread.
ORM	Prisma	Type-safe query access with built-in `$transaction` guarantees for double-entry ledger integrity.
Database	SQLite (`nexus.db`)	Local filesystem storage. Zero cloud dependency. Guarantees offline capability and complete data privacy.
Background compute	Node.js `worker_threads`	Offloads report generation and analytics scoring from the UI thread, keeping the interface responsive during heavy computations.
Scheduling	`node-cron`	Weekly adaptation evaluator, daily budget checks, habit log resets.
Adaptive state	Flat JSON file	Written to `userData/adaptive_config/feature-flags.local` via atomic temporary-file-and-rename write strategy.

Table 9: NEXUS Adaptive tech stack

C. Prototype C: Conduit Retrofit (Full-Stack Web App)

Layer	Technology	Notes
Frontend	React + Vite + JavaScript	Pre-existing codebase. Adaptive layer added via tracker, FeatureGate component, useFlipper hook.
Backend	Express.js (Node.js)	Pre-existing. flipperGuard middleware added for gated routes. Adaptive controller added for track/restore endpoints.
ORM	Sequelize	Pre-existing. `adaptiveSchema` JSON column added to User model. `FeatureUsageStat` model added with composite primary key.
Database	PostgreSQL	Upsert via `INSERT ... ON CONFLICT DO UPDATE` for atomic usage count increments.
Adaptive state	JSON column in Users table	Per-user. Read alongside login payload. Sanitized on every read.
Evaluation trigger	Login / currentUser hooks	`setImmediate` async; 7-day minimum gap enforced.

Table 10: Conduit retrofit tech stack

VII. Team Roles

Our team divided Release 2 responsibilities across the knowledgebase design, the two NEXUS prototypes, the Conduit retrofit, and documentation.

Pranav Tej (CS24B057) — Knowledgebase Developer

Responsibilities: Authoring the core knowledgebase entry point files and the feature assessment module, and conducting the final end-to-end review of the complete knowledgebase before submission.

Release 2 Contributions: Authored `00_Context_and_Goals/00_goal.md` (the framework entry point explaining the Observer Paradox, the four-module architecture, and the Feature ID contract, with the hard traversal directive that gates all downstream agent reads) and `00_Context_and_Goals/01_developer_context.md` (the context intake agent that resolves the seven project dimensions and produces the Project Context Summary JSON). Authored `02_Assessment_and_Tagging/01_feature_tagging.md` (the feature audit instruction file defining the Feature ID contract, the five taggable categories, the locked-feature specification, the Feature Registry schema, and the three required output artifacts). Performed the final review pass across all knowledgebase files, validating cross-file references, correcting inconsistencies between the router matrix and pattern file names, and verifying that the traversal protocol produces no circular reads.

Ankit Sinha (CS24B056) — Research Lead & Knowledgebase Architect

Responsibilities: Designing and authoring the full agent-optimized knowledgebase, validating the traversal protocol against both prototype codebases, and producing all Release 2 documentation.

Release 2 Contributions: Authored all files in `00_Context_and_Goals`, `01_Architecture`, `02_Assessment_and_Tagging`, and `04_Memory_and_Context`. Designed the Developer Context Intake protocol and the Project Context Summary JSON schema. Conducted the Feature Tagging audit for both the NEXUS and Conduit codebases, producing the Feature Registry, Initial Metadata Schema, and Tagging Checklist artifacts. Authored this report.

Karthik (CS24B046) — Prototype B Developer (NEXUS Adaptive)

Responsibilities: Taking Bhargav's NEXUS baseline and applying the full AdaptiveFlow framework across all twelve architectural layers using the knowledgebase as the implementation guide. Used AI coding agents (Cursor, Antigravity) extensively throughout development.

Release 2 Contributions: Implemented the complete NEXUS adaptive layer (Prototype B): the `AdaptiveProvider.tsx` React context and `useFlipper` hook, the `FeatureRoute` and `FeatureGate` gated routing components, sidebar and widget-level gating in `Layout.tsx` and `Dashboard.tsx`, the `HeavyChart.tsx` lazy-import gate, the `FeatureUsageLog` Prisma model with composite unique key, the batched ingestion endpoint (`POST /api/feature-usage/batch`), the 14-day rolling window aggregation query, the adaptation evaluation function in `server/adaptive/adaptationService`, the weekly cron evaluation trigger, and the Settings-page restore interface (`HiddenFeatureNotice.tsx`).

Bhargav (CS24B028) — Prototype A Developer (NEXUS Baseline)

Responsibilities: Building the full NEXUS application *without* the adaptive framework — all ten feature modules, the Express backend, the Prisma/SQLite database schema, the background worker thread system, and the CRON infrastructure — providing a clean, feature-complete baseline that Karthik could then retrofit with the full adaptive layer. Used AI coding agents (Cursor, Antigravity) extensively throughout development.

Release 2 Contributions: Developed the complete NEXUS baseline codebase (Prototype A): the React + Vite glassmorphic frontend with all ten page components (Dashboard, Accounts & Transactions, Tasks, Habits, Notes, Journal, Budgets, Goals, Investments, Reports, Analytics), the Express backend with all API route suites, the Prisma schema with all models and `$transaction` guarantees, the `report.worker.ts` and `analytics.worker.ts` background workers, and the CRON schedule for daily budget checks and habit log resets. Delivered a well-structured, modular codebase explicitly designed for Karthik to apply the adaptive framework on top without requiring structural refactoring.

Nilanjan Singh (CS24B029) — Prototype C Developer (Conduit Retrofit)

Responsibilities: Retrofitting the existing Conduit RealWorld full-stack application with the AdaptiveFlow framework, implementing the client-side Monitor module, the Electron-specific schema persistence layer for NEXUS, and all feature invocation tagging across both codebases. Used AI coding agents (Cursor, Antigravity) throughout development.

Release 2 Contributions: Built and integrated the complete Conduit adaptive layer (Prototype C): the frontend tracker (`frontend/src/adaptive/tracker.js`) with document-click delegation, per-feature throttling, queue capping, timed 30-second flush, and `navigator.sendBeacon` lifecycle flush; the `FeatureGate` component and `useFlipper` hook for Conduit's React frontend; the `flipperGuard` backend middleware for all gated Conduit routes; the `adaptiveSchema` JSON column migration on the Sequelize `User` model; the per-user schema read/write/sanitize flow; and the login/currentUser async adaptation trigger. Also implemented the Electron-side `readAdaptiveSchema` and atomic-write `writeAdaptiveSchema` IPC handlers for NEXUS, and tagged all feature invocation points across both codebases with `data-feature-id` attributes and `useFeatureLog` hook calls per the Tagging Checklist.

VIII. AI Usage Declaration

A. Ankit

Knowledgebase authoring — AI assistance: approximately 70%

Tool: Gemini

All knowledgebase files were AI-generated via Gemini. The structure, the traversal protocol design, the numbered-prefix directory convention, the router-as-orchestrator pattern, the seven context dimensions, the naming conventions (`ID_[DOMAIN]_[CAPABILITY]`), the anti-pattern lists, and the decision to separate memory into Spatial and Episodic domains were all conceived and directed manually. AI was used to produce the actual file content from those human-designed specifications. All strategic architecture decisions, routing logic, and framework philosophy were human-authored; the AI was a content generator, not a design authority.

B. Pranav

Knowledgebase file authoring — AI assistance: 75%

Tool: Gemini

Gemini was used to generate the full content of the three assigned knowledgebase files from human-designed specifications. The framework philosophy, the traversal directives, the intake questionnaire structure, and the feature category definitions were authored manually; AI generated the verbose instructional prose and the JSON schema examples within each file.

Prompt sequence:

Entry 1 "Write the `00_goal.md` entry point for an agent-optimized adaptive software knowledgebase. It must explain the Observer Paradox (why runtime monitors are counterproductive), describe the four core modules (Monitor, Statistics, Flipper, Evaluator), define the Feature ID contract, and end with a hard directive telling the agent not to read any other file yet and to proceed to the router. Write in the style of a system directive." Outcome: Useful. The hard traversal directive wording was manually revised for precision.

Entry 2 "Write `01_developer_context.md` as a context intake agent. It must resolve seven project dimensions: application type, frontend stack, backend stack, database and ORM, identity model, performance constraints, and codebase state. Produce a structured Project Context Summary JSON schema at the end that will be attached to all downstream agent calls." Outcome: Useful. The seven-dimension list and the JSON field names were manually specified before the prompt.

Entry 3 "Write `02_Assessment_and_Tagging/01_feature_tagging.md` for the knowledgebase. It must define the Feature ID as a contract between four decoupled systems, specify the `ID_[DOMAIN]_[CAPABILITY]` naming convention, define the five taggable feature categories with examples, explain locked features, define the Feature Registry JSON schema, list strict anti-patterns, and specify the three output artifacts. Write it as a system directive for an AI agent." Outcome: Useful as a structural scaffold. Anti-patterns and Single Responsibility Test were manually rewritten.

C. Karthik

NEXUS adaptive layer development — AI assistance: 85%

Tools: Cursor, Antigravity

Karthik used Cursor and Antigravity AI coding agents throughout the development of Prototype B, working from Bhargav's baseline and the knowledgebase as the implementation guide. Agents were used to generate the frontend Flipper context, gated routing components, the usage statistics pipeline, and the weekly adaptation evaluator.

Entry 1 Read `01_Architecture/02_flipper_module.md` from the knowledgebase and asked Cursor to implement the `AdaptiveProvider` React context and `useFlipper` hook for the existing NEXUS app. Outcome: Useful. Generated the provider and hook correctly.

Entry 2 Used the knowledgebase `03_Code_Patterns/01_frontend_dom.md` pattern and asked Cursor to create a `FeatureRoute` gating component for all NEXUS page routes. Outcome: Partially used. Static imports had to be corrected to dynamic `React.lazy` wrapping.

Entry 3 Asked Antigravity to implement the full usage tracking pipeline: `FeatureUsageLog` Prisma model, batched ingestion endpoint, and 14-day rolling-window aggregation query, following the statistics module spec from the knowledgebase. Outcome: Useful with minor corrections to the `groupBy` where-clause nesting.

D. Bhargav

NEXUS baseline development — AI assistance: 90%

Tools: Cursor, Antigravity

Bhargav used Cursor and Antigravity AI coding agents throughout the development of Prototype A. These agents were used to accelerate scaffolding, generate boilerplate, and implement complex multi-table Prisma transactions for the non-adaptive NEXUS application.

Entry 1 Asked Cursor to scaffold a full Electron + React + Vite + TypeScript + Express + Prisma + SQLite project with an IPC bridge between the renderer and the local Express backend. Outcome: Useful. Port negotiation between Electron and Express required manual correction.

Entry 2 Asked Cursor to implement the double-entry transaction system: creating a transaction should atomically insert a `Transaction` record and adjust the linked `Account` balance via Prisma `$transaction`. Outcome: Useful. Concurrent transaction race condition required manual fix.

Entry 3 Asked Antigravity to create a `report.worker.ts` Node.js worker thread that queries `Transaction`, `Account`, and `Investment` tables, categorises by INCOME/EXPENSE, and returns a structured cash-flow result without re-initialising Prisma on every invocation. Outcome: Useful. Prisma client instantiation inside the worker required correction.

E. Nilanjan

Conduit retrofit and Electron integration — AI assistance: 80%

Tools: Cursor, Antigravity

Nilanjan used Cursor and Antigravity AI coding agents throughout Prototype C (Conduit retrofit) and the Electron schema persistence work. These agents accelerated the tracker implementation, the Sequelize model extension, and the IPC handler generation.

Entry 1 "Build a TypeScript class `FeatureTracker` with an in-memory event queue. When `track(featureId)` is called, push {`f_id, ts: Date.now()`} to the queue. Throttle: skip if the same `f_id` was pushed within the last 1000 ms. Flush on a 30-second interval. Flush on `visibilitychange` to hidden." Outcome: Useful. The throttle implementation needed adjustment to use a per-feature last-seen timestamp rather than a single global debounce.

Entry 2 "In the Electron main process, implement `readAdaptiveSchema()` that reads the JSON file from `app.getPath('userData')`, parses it, validates the shape (schemaVersion, features, lockedFeatures), and returns a safe default if parsing fails. Implement `writeAdaptiveSchema(schema)` that writes to a temp file then atomically renames to the real path." Outcome: Useful. Added the IPC handler exposure (`ipcMain.handle`) independently.

Entry 3 "For the Conduit retrofit, generate the `tracker.js` equivalent in plain JavaScript with the global document-click delegate approach. Tag elements via `data-feature-id`. Add `navigator.sendBeacon` as the flush strategy on `beforeunload` and `visibilitychange`." Outcome: Useful.

IX. Prompts

A. Ankit

Prompt 1 **Tool:** Gemini
Prompt: "Fetch context from 'jake van clief' from another chat. We will be using

his method for this. You know what we want to build. Now suggest a knowledgebase structure that will help llm agents to provide value without burning millions of tokens with them scanning through the entire knowledgebase also the knowledgebase should provide info on how exactly to develop or implement or refactor to adaptive softwares that we aim THE FOLLOWING IS MY IDEA, IF YOU THINK ITS BAD AND THERE EXISTS A BETTER WAY THEN SUGGEST # Guideline knowledgebase structure (goal: agent friendly knowledgebase that prevents token waste) Root: Adaptive software knowledgebase folder Top level files: goal.md: Explains the whole toggleflagging method overview and how exactly it saves compute and resources. Explains what metafiles (yaml/json bool file it needs to maintain) and flipper module it needs to maintain and develop. At the end of the file it instructs user/agent to go to flipper.md. flipper.md: instructions (an idea) on how to implement flipper module and flag file maintaining module and statistics module. redirect to redirector.md. Redirector.md: Has code patterns mapped to specific changing instructions and ideas for implementation of flipper and benefits. Also maps to the top level .md files if the model ever needs to refer again. SUBfolder: Codepattern_Changes has all files eg codepattern_changes/api_Routes.md – instructs on how backend logic on api routes should be featureflagged. eg it maintains a metadata col on what info pieces the user needs and avoids computation and fetching of these data pieces. codepattern_changes/webpage.md – instructs on how to implement flagging for toggle of DOM element rendering"
Purpose: Validate new approach's raw idea.
Outcome: Validated the structure; AI confirmed numbered-prefix approach and suggested the router-as-orchestrator pattern.

Prompt 2 **Tool:** Gemini
Prompt: "What other code patterns do you think we need"
Purpose: Prompt the model to put out suggestions for additional code pattern files.
Outcome: Useful. Suggested middleware, asset manager, db query fragmentation, package init, and memory cleanup patterns.

Prompt 3 **Tool:** Gemini
Prompt: "# Changes: (make sure implementation is lightweight) NEW architecture/statistics_module.md – lies on the backend, fed data from monitor_module. NEW architecture/monitor_module.md – lies on the frontend. Specify in metadata.md that filedata needs to be stored with userdata in database. Maybe an option where the dev inserts context about the usage and goals of his current codebase if any. NEW codepatterns: middleware.md, asset_manager_and_cache.md (for certain users you don't need to preload certain assets), db_query_fragmentation.md (suppose the guy never needs a profilepic and a table join is done to get a profile pic. now we can skip that costly join. so fragment code so that the join is only performed when necessary), package_init.md (some heavy packages are heavy even when initialized like chart.js in frontend. don't do this for backend since it's common for users. featureflag them so it's never imported since it's not going to be used). MODIFY codepatterns: stray_references.md EXAMPLE: once you remove an element from DOM tree, there may be stray references as event listeners. need to guide to featureflag those too with the same flag. NOW GIVE A NEW FOLDER STRUCTURE `adaptive_software_kb/` with the proposed expanded layout including 00_Context_and_Goals, 01_Architecture (with router.md, flipper_module.md, metafile_schema.md), 02_Assessment_and_Tagging,

03_Code_Patterns (8 files), and 04_Memory_and_Context."
Purpose: Make a new folder structure with the new changes suggested by both the model and also teammates.
Outcome: Produced the final knowledgebase directory structure used in this release.

Prompt 4 **Tool:** Gemini
Prompt: "Make all files from 01_architecture make sure it is generalised to all kinds of applications – apps, websites, desktop uis, etc"
Purpose: Generate the 01_Architecture files in a stack-agnostic way.
Outcome: Useful. Generated the initial drafts for router.md, flipper_module.md, and metafile_schema.md covering web, desktop, and mobile stacks.

Prompt 5 **Tool:** Gemini
Prompt: "make a very detailed router.md"
Purpose: Generate the routing orchestrator file with a comprehensive 5-category classification matrix.
Outcome: Useful. The routing matrix and trigger-keyword mappings were refined manually.

Prompt 6 **Tool:** Gemini
Prompt: "now statistics module >100 lines"
Purpose: Generate a detailed statistics module specification exceeding 100 lines.
Outcome: Useful. Sections on batching strategy and upsert atomicity were manually expanded.

Prompt 7 **Tool:** Gemini
Prompt: "flipper logic >100 lines"
Purpose: Generate the Flipper module implementation spec exceeding 100 lines covering all stacks.
Outcome: Useful. The fail-open default behavior section was manually authored for clarity.

Prompt 8 **Tool:** Gemini
Prompt: "metadata schema >100 lines"
Purpose: Generate the metadata schema contract file exceeding 100 lines with all field constraints and persistence rules.
Outcome: Useful. Atomic write strategy and sanitization invariants were manually validated.

Prompt 9 **Tool:** Gemini
Prompt: "do codepatterns/01, 07, 08"
Purpose: Generate code pattern files for frontend DOM gating (01), package imports (07), and memory cleanup (08).
Outcome: Useful. The WebGL context disposal section in 08 and the React.lazy contract in 07 required manual correction.

Prompt 10 **Tool:** Gemini
Prompt: "i want these features too. 04_codebase_map.md (Spatial Memory) Purpose: O(1) context retrieval for file locations. How it works: Agents waste massive amounts of tokens searching through directories or reading tree outputs. This file is a living index of the application's architecture. Contents: A mapped registry like: Feature: User

Auth -> /src/controllers/auth.ts, Feature: Adaptive Flipper -> /src/lib/flipper.ts. When subagents create new files or move things, they are instructed to update the codebase_map.md. The orchestrator uses this map to instantly know which files to inject into a subagent's prompt context. 05_action_log.md (Episodic Memory) Purpose: Prevents infinite loops and duplicated work. How it works: Every time a subagent successfully completes a sub-task (e.g., 'Tagged all DOM elements in Nav.jsx with Feature IDs'), it logs the timestamp, the action, and the outcome here. Contents: An append-only ledger of completed milestones. If the orchestrator loses its train of thought or the system restarts, it reads the action log to resume exactly where it left off, rather than starting the entire adaptation process from scratch."
Purpose: Insert a new idea into the context memory and generate the new structure to document the Spatial and Episodic memory system.
Outcome: Useful. Led directly to the 04_Memory_and_Context folder design with codebase_map.md and action_log.md.

Prompt 11 **Tool:** Gemini
Prompt: "give me a prompt so an agent can implement this in the existing knowledgebase. be very detailed and also discuss the role of each file and also tell the agent to fill it up, and where should the instructions for how to use these memory files even be in the knowledgebase. (maybe in goal.md?)"
Purpose: Generate an implementation prompt for an agent to populate the memory system files.
Outcome: Useful. Produced the agent bootstrap prompt and confirmed that memory protocol instructions belong in `00_memory_protocol.md` rather than in `goal.md`.

B. Pranav

Prompt 1 **Tool:** Gemini
Prompt: "Given this `cron.ts` file for a Node.js/Express backend with Prisma and SQLite, add a weekly adaptation evaluator function. It must: read the feature-flags JSON file at a given path, query the `FeatureUsageStat` table for the sum of `usageCount` per `featureId` where `timeBucket` is within the last 14 days, skip any featureId in the `lockedFeatures` array, flip features below their threshold to false in the schema object, update `lastEvaluated`, and write the result atomically to disk using a temp-file-and-rename pattern."
Purpose: Generating the adaptation evaluator for NEXUS.
Outcome: Useful. Prisma `groupBy` query required manual correction.

Prompt 2 **Tool:** Gemini
Prompt: "Create an Express middleware called `flipperGuard` that takes a `featureId` string parameter. It should read `req.adaptiveSchema` (attached by a preceding auth middleware), call `flipper.isActive(featureId, schema)`, and either call `next()` or return 403 with `{ error: 'Feature disabled', featureId }`."
Purpose: Backend route gating middleware for NEXUS.
Outcome: Useful.

C. Karthik

Prompt 1 **Tool:** Cursor
Prompt: "Hey so I have this Electron + React + Express app (NEXUS) that my

teammate built. I read the knowledgebase file `02_flipper_module.md` and it says I need to add an `AdaptiveProvider` and a `useFlipper` hook. I have no idea where to start. Here is my `App.tsx`. Can you add the provider wrapping and the hook for me? The schema should come from Electron IPC."
Purpose: Adding the Flipper React context to the existing NEXUS baseline.
Outcome: Useful. Generated the provider and hook correctly on first attempt.

Prompt 2 **Tool:** Cursor
Prompt: "Ok I read `03_Code_Patterns/01_frontend_dom.md` and it says I need to lazy-gate my routes. I currently have static imports at the top of `App.tsx` for every page. How do I change this so that if `useFlipper` returns false for a feature, the page bundle is never loaded at all? I do not want a redirect, I want the JS to literally never download."
Purpose: Gating NEXUS page routes with lazy-load following the knowledgebase pattern.
Outcome: Partially useful. The first output still used static imports at the top. Had to ask again specifically for `React.lazy` wrapping.

Prompt 3 **Tool:** Antigravity
Prompt: "I followed `03_Code_Patterns/02_backend_api.md` and now I need the `flipperGuard` middleware on the Express side too. Here is my `server/app.ts`. The schema is already attached to `req.adaptiveSchema` by the auth middleware. Just add the guard to the reports and analytics route groups. Also add the restore endpoint that lets the frontend PUT an updated schema back."
Purpose: Adding backend route gating and schema restore endpoint.
Outcome: Useful. Minor fix needed for the PUT body validation.

Prompt 4 **Tool:** Cursor
Prompt: "The knowledgebase says I need a usage tracking module. I read `02_flipper_module.md` stats section. I need a `FeatureUsageLog` Prisma model, a batched `POST /api/feature-usage/batch` endpoint, and a 14-day rolling aggregation query for the weekly evaluator. Here is my current `schema.prisma`. Can you add all of this?"
Purpose: Adding the full statistics pipeline to the NEXUS adaptive layer.
Outcome: Useful. The `groupBy` where-clause nesting needed a small correction.

D. Bhargav

Prompt 1 **Tool:** Cursor
Prompt: "Scaffold a full Electron + React + Vite + TypeScript + Express + Prisma + SQLite project. The folder structure should have an `electron/` main process, a `src/` React frontend, and a `server/` Express backend on a dynamic port. The Express backend should connect to a local SQLite database via Prisma, stored in the Electron userData path. Include a basic IPC bridge so the renderer can call the backend via HTTP."
Purpose: Generating the initial NEXUS project structure for Prototype A.
Outcome: Useful. Port negotiation between Electron and Express required manual correction.

Prompt 2 **Tool:** Cursor
Prompt: "Implement the double-entry transaction system in Prisma + Express.

Creating a transaction should atomically: insert a record into the `Transaction` table and adjust the balance on the linked `Account` row using `$transaction`. Return 400 if the account does not exist. The Prisma schema should enforce the relationship."
Purpose: Core financial ledger integrity for NEXUS accounts module.
Outcome: Useful. Race condition edge case in concurrent transactions required manual fix.

Prompt 3 **Tool:** Antigravity
Prompt: "Create a `report.worker.ts` Node.js worker thread. It should receive a `WorkerMessage` with `userId` and a date range. It queries `Transaction`, `Account`, and `Investment` tables via Prisma, categorises transactions into INCOME and EXPENSE, computes cash flow, and returns a structured result object. Worker must not re-initialise Prisma on every invocation."
Purpose: Offloading heavy report aggregation from the main thread.
Outcome: Useful. Prisma client instantiation inside the worker required correction.

E. Nilanjan

Prompt 1 **Tool:** Cursor
Prompt: "I have to make the Conduit app adaptive using this knowledgebase. I read `01_feature_tagging.md` and listed the features I want to gate: popular tags sidebar, personal feed tab, favorited articles tab, and comments section. Now I need to add the tracker. The knowledgebase says I need a JS class that listens for clicks on elements with `data-feature-id`, throttles to once per 1000ms per feature, queues events, and flushes every 30 seconds or on tab hidden. Can you write this for the Conduit React frontend?"
Purpose: Adding the Monitor module tracker to Conduit following the knowledgebase spec.
Outcome: Useful. Per-feature throttle logic required correction.

Prompt 2 **Tool:** Antigravity
Prompt: "Ok now I need to add the feature gating to the Conduit React frontend. I read `03_Code_Patterns/01_frontend_dom.md`. I need a `FeatureGate` component that checks `useFlipper` and renders null if the feature is off. The schema should come from the login API response. Here is my `App.jsx` and `Login.jsx`. Can you wire this up?"
Purpose: Frontend gating for Conduit retrofit following the knowledgebase DOM pattern.
Outcome: Useful.

Prompt 3 **Tool:** Cursor
Prompt: "The knowledgebase `02_flipper_module.md` says I need to store the adaptive schema per user. Conduit uses Sequelize and PostgreSQL. I need to add an `adaptiveSchema` JSON column to the User model, write a migration, and add a sanitize helper that returns a safe default if the stored JSON is broken. Here is my current User model. Can you add this?"
Purpose: Per-user schema persistence for Conduit following the knowledgebase storage spec.
Outcome: Useful. Sequelize `DataTypes.JSON` serialization needed manual verification.

Prompt 4 **Tool:** Antigravity
Prompt: "Last step, I need backend gating. I read `03_Code_Patterns/02_backend_api.md`. I need a `flipperGuard` middleware for Express that reads `req.user.adaptiveSchema`, checks if the feature is active, and returns 403 if not. Then I need to add it to the personal feed route and the comments route. Here is my `router.js`. Can you add the middleware and the guarded routes?"
Purpose: Backend dual-layer gating for Conduit following the knowledgebase middleware pattern.
Outcome: Useful.

X. Milestones & Timeline

Phase	Milestone & Deliverables	Target Date	Assigned
Release 1	Architecture validation and UI prototypes: MAPE-K engine, three distinct adaptation prototypes over the MS Excel clone testbed. Proposal and Release 1 report.	Mar 17, 2026	All
Sprint 1	Backend infrastructure: core feature-flag JSON parser, Flipper singleton, statistics ingestion endpoint, feature ID tagging checklist for NEXUS.	Apr 01, 2026	Pranav, Ankit, Nilanjan
Release 2	Framework abstraction and full adaptive implementation: knowledgebase complete and committed; Prototype A (NEXUS baseline) complete; Prototype B (NEXUS adaptive) complete with all 12 adaptation layers integrated; Prototype C (Conduit retrofit) complete with dual-layer gating and per-user schema.	Apr 15, 2026	All
Sprint 2	Performance evaluation: RAM/CPU profiling before and after adaptation for both NEXUS and Conduit; user testing for task completion time and cognitive load; quantitative analysis of feature usage patterns across prototype sessions.	Apr 29, 2026	Karthik, Bhargav, Ankit
Final	Code freeze and polish: final bug resolution, knowledgebase review, full `03_Code_Patterns` file set (currently 8 pattern files) verified against both prototypes, production of the project demonstration video, comprehensive documentation.	May 10, 2026	All

Table 11: Project milestones and timeline

XI. Goals for Final Release

A. Framework Goals

- Complete all eight `03_Code_Patterns` implementation files, verified against at least one real prototype each.
- Publish the final `04_Memory_and_Context` working memory system with populated orchestrator learnings and subagent error ledger entries accumulated during the two prototype builds.
- Add a mobile platform chapter to the knowledgebase covering React Native's `AsyncStorage`

schema persistence and iOS/Android lifecycle flush hooks.

B. Prototype Goals

- **Prototype B (NEXUS Adaptive):** Complete production-grade implementation across all 12 adaptation layers listed in the architecture table. Produce quantitative measurements of RAM consumption and startup time with multiple adaptive profiles (all features on vs. a realistic low-usage profile).
- **Prototype C (Conduit):** Complete the full dual-layer implementation. Conduct user testing with simulated divergent usage patterns to validate that the evaluator correctly identifies and hides low-usage features without affecting high-usage ones.
- **Agent Comparison Study:** A formal comparison of code generated by a Codex-class agent *with* and *without* the knowledgebase for the same implementation task. Evaluate: correctness of Feature ID consistency, presence of Observer Paradox violations, adherence to the lazy-loading contract, and correctness of the atomic write pattern.

C. Research Goals

- Quantify and report: the RAM savings achieved by disabling the Recharts heavy chart import (`ID_DASHBOARD_ANALYTICS_CHART`) and the report worker (`ID_REPORTS_GENERATE`) in NEXUS.
- Quantify and report: the reduction in Express route overhead achieved by the `flipperGuard` middleware blocking disabled routes before any database access in Conduit.
- Validate the agent-traversal claim: demonstrate with token-count logs that an agent using the knowledgebase consumes fewer tokens than one without it for an equivalent implementation task.